

```
bits_output.append(bits_next.copy())
random_output.append(next)
return(bits_output, random_output)
```

In this case, we run the `while` loop until we expect the series to repeat. Since there are $2^3 = 8$ possible states for our bits list, we can expect a period of at most 8. Actually, LFSRs typically cannot output a full set of zeros, so in practice we expect a period of at most $2^3 - 1 = 7$. We can run the following code to find all possible outputs and check the period:

```
bitslist = feedback_shift_list([1,1,1])[0]
```

Sure enough, the output that we stored in `bitslist` is

```
[1, 1, 1], [0, 1, 1], [0, 0, 1], [1, 0, 0], [0, 1, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]
```

We can see that our LFSR outputs all seven possible bit-strings that are not all 0s. We have a full-period LFSR, and also one that shows a uniform distribution of outputs. If we use more input bits, the maximum possible period grows exponentially: with 10 bits, the maximum possible period is $2^{10} - 1 = 1023$, and with only 20 bits, it is $2^{20} - 1 = 1,048,575$.

We can check the list of pseudorandom bits that our simple LFSR generates with the following:

```
pseudorandom_bits = feedback_shift_list([1,1,1])[1]
```

The output that we stored in `pseudorandom_bits` looks reasonably random given how simple our LFSR and its input are:

```
[1, 1, 1, 0, 0, 1, 0]
```

LFSRs are used to generate pseudorandom numbers in a variety of applications, including white noise. We present them here to give you a taste of advanced PRNGs. The most widely used PRNG in practice today is the *Mersenne Twister*, which is a modified, generalized feedback shift register—essentially a much more convoluted version of the LFSR presented here. If you continue to progress in your study of PRNGs, you will find a great deal of convolution and advanced mathematics, but all of it